

Privacy-Preserving Lightweight Time-Series Anomaly Detection for Resource-Limited Industrial IoT Edge Devices

Lei Chen^{ID}, Member, IEEE, Yepeng Xu^{ID}, Ming Li, Bowen Hu^{ID}, Haomiao Guo, and Zhaohua Liu^{ID}, Senior Member, IEEE

Abstract—Identifying anomalies directly on edge devices rather than in the cloud, known as edge computing, is essential for Industry 4.0. However, the limited computing and storage resources on edge devices render traditional cloud-based anomaly detection models ineffective. To solve this issue, a privacy-preserving lightweight time-series anomaly detection model, named PPLAD, is proposed for resource-limited industrial Internet of Things (IoT) edge devices via global and local similarity discrepancy. *First*, PPLAD directly uses data similarity instead of raw data as model input, to achieve privacy-preserving. *Second*, PPLAD applies trainable Gaussian distribution rather than deep neural network as model structure, to achieve high timeliness and low cost. Specifically, PPLAD constructs a trainable Gaussian distribution with only one parameter for each timestamp to model its similarity with neighbors. *Third*, a global and local adversarial learning strategy is developed to amplify the discrepancy between local similarity and global similarity for each timestamp. Finally, the discrepancy is used to accurately identify timestamp-level anomalies. To the best of authors' knowledge, this is the first work to build an industrial anomaly detection model using only learnable Gaussian distributions. Extensive experiments on eight public industrial multisensor datasets and three edge devices demonstrate that PPLAD outperforms several state-of-the-art models.

Index Terms—Data privacy-preserving, edge devices, industrial Internet of Things, lightweight anomaly detection, resource-limited, similarity discrepancy.

I. INTRODUCTION

ANOMALY detection is crucial and widely concerned for costly industrial scenarios [1]. In the past, anomaly

Received 26 October 2024; revised 10 January 2025; accepted 20 January 2025. Date of publication 3 March 2025; date of current version 23 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62103143, in part by the Hunan Provincial Natural Science Foundation of China under Grant 2024JJ5162 and Grant 2024JJ7179, and in part by the Scientific Research Fund of Hunan Provincial Education Department under Grant 22B0471. Paper no. TII-24-5638. (Corresponding author: Lei Chen.)

The authors are with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan 411201, China (e-mail: chenlei@hnust.edu.cn; xuyepeng@mail.hnust.edu.cn; minglee@hnust.edu.cn; hubowen@mail.hnust.edu.cn; guohaomiao@mail.hnust.edu.cn; zhaohua.liu@hnust.edu.cn).

The source code of PPLAD is available at <https://github.com/infogroup502/PPLAD>.

Digital Object Identifier 10.1109/TII.2025.3538127

detection models are often deployed on the cloud with powerful resources [2]. In this case, edge devices periodically upload the collected data to the cloud. The detection model in the cloud identifies anomalies and then returns the results to edge devices. These models prioritize enhancing feature representation and model accuracy (ACC) by building deep neural networks with complex structures and huge parameters. Therefore, deep neural network-based models are the dominant focus of current research. However, driven by fine-grained industrial smart controls, an increasing number of Internet of Things (IoT) sensors are being used to perform more intensive data collection and information perception. As a result, the volume of data is growing rapidly, which causes substantial communication overhead between edge devices and the cloud. Moreover, the huge data volume further makes anomaly detection in the cloud result in high latency and high costs. Therefore, identifying anomalies directly on edge devices rather than in the cloud, also known as edge computing, is a critical requirement for Industry 4.0 [3]. However, limited computing and storage resources on edge devices make the current popular deep neural network-based models powerless. Specifically, there are two major challenges for existing models on resource-limited edge devices [4].

A. Challenges

1) *Balancing ACC, Timeliness, and Cost-Efficiency is Challenging*: Anomaly detection on resource-limited edge devices must simultaneously meet high ACC, high timeliness, and low-resource consumption [5]. However, these goals are often mutually conflicting and difficult to balance. For example, to get high ACC, detection models often need to have strong representational capabilities by building deep neural networks with complex structures and huge parameters [6], [7], [8]. In this case, these models demand long processing times and consume substantial computational and storage resources. Conversely, to achieve high timeliness, detection models usually lightweight models with shallow structures and few parameters. Although these models only consume less computational and storage resources, but they have poor feature capture capability and unsatisfactory detection ACC.

2) *Data Privacy is Not Secure and Easily Exposed*: In industrial scenarios, edge devices continuously collect sensor temporal signals from the production process. These sensor data contains highly valuable information and has strong privacy. Currently, existing anomaly detection models for IoT edge devices directly take raw sensor signals as input data. However,

IoT edge devices are often insecure [9], [10] and lack powerful protection mechanisms. In this case, external attacks can easily steal the raw sensor signal data, resulting in a privacy leakage. In more severe cases, attackers may compromise edge devices and disrupt production processes.

B. Existing Solutions

As for the first challenge, some works [11], [12] use deep neural networks as prototypes, employing parallelization or compression techniques to reduce network layers and redundant parameters, ultimately creating lightweight models that balance ACC and timeliness. However, these models still have numerous parameters and a deep structure, resulting in unsatisfactory resource consumption and processing time. The other works rely on shallow machine learning or statistical models, combined with other techniques or customized ACC optimization strategies, to build hybrid lightweight models [13], [14], [15] that balance ACC and timeliness. However, these models may be unstable when encountering multisensor temporal signals with complex feature patterns.

As for the second challenge, existing solutions [16], [17] employ some security mechanisms to guarantee data privacy. However, these models require additional encryption and decryption processing, which results in more computational, communication, and storage resource consumption and processing time. In certain industrial scenarios, some privacy-preserving technologies may not even be feasible on resource-efficient edge devices.

In summary, performing anomaly detection with high speed, high ACC, and low-resource consumption on resource-limited edge devices remains a significant challenge.

C. New Insights

To fill this gap, this article tries to design a lightweight anomaly detection model to achieve the desired goals. Specifically, two key bottlenecks need to be solved.

The first bottleneck is how to build a lightweight model with high ACC, high timeliness, and low-resource consumption. Anomalies in industrial IoT systems occur infrequently, typically spanning multiple timestamps, and exhibit distinct characteristics compared to normal data, as shown in Fig. 1(a). It implies that an abnormal timestamp is only similar to its adjacent neighbors (local neighbors) and not to its distant neighbors (global neighbors). On the contrary, a normal timestamp is similar not only to its local neighbors but also to its global neighbors. Hence, the local-global similarity discrepancy offers a useful criterion for distinguishing anomalies from normal timestamps, as shown in Fig. 1(b).

Driven by this observation, we try to set up a tiny trainable Gaussian distribution with only one parameter for each timestamp, so as to learn its local and global similarities. Meanwhile, we design an adversarial learning mechanism to amplify the discrepancy between global and local similarities for each timestamp. This mechanism works by simultaneously maximizing its local and global similarities. In this scheme, the lightweight trainable Gaussian distribution with only a parameter ensures high timeliness and low consumption, while the adversarial learning mechanism provides high ACC.

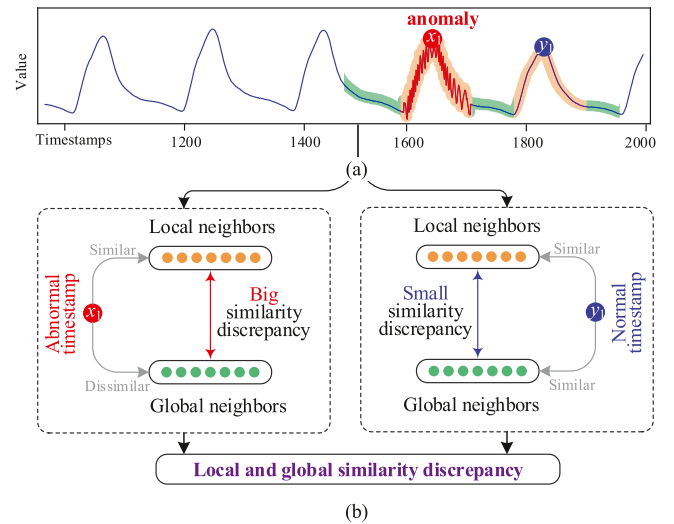


Fig. 1. Illustration of the local and global similarity discrepancy. (a) Anomaly in real-world UCR dataset. (b) Anomaly detection driven by similarity discrepancy.

The second bottleneck is how to achieve flexible privacy-preserving for multisensor temporal data without incurring additional overhead. A straightforward scheme is to use data similarity as an initial input rather than raw privacy data. In this scheme, the initial similarity computation can either be outsourced to a third-party security interface or flexibly integrated with other models.

Based on these insights, a privacy-preserving lightweight anomaly detection model, named PPLAD, is proposed for resource-limited industrial IoT edge devices. First, PPLAD uses data similarity as an initial input. Second, PPLAD employs a tiny trainable Gaussian distribution for each timestamp to learn both local and global similarities. Finally, PPLAD designs an adversarial learning mechanism to amplify the discrepancy between global and local similarities and use this discrepancy to detect anomalies.

D. Contributions

The main contributions of this article are listed as follows.

- 1) A learnable Gaussian distribution-based lightweight anomaly detection model, named PPLAD, is proposed for resource-limited industrial IoT edge devices. Driven by the global and local similarity discrepancy, PPLAD employs the tiny learning structure and customized adversarial mechanism to achieve high timeliness, high ACC, and low-resource consumption. To the best of authors' knowledge, this is the first work to build an industrial anomaly detection model using only learnable Gaussian distributions.
- 2) PPLAD utilizes data similarity as the initial input, rather than raw multisensor privacy data. This approach not only protects data privacy but also enables flexible integration with other models.
- 3) Extensive experiments on eight public industrial multisensor datasets and three edge devices (Thick PC, Thin Jetson Xavier NX, Mirco Raspberry Pi 4b) demonstrate that PPLAD outperforms several state-of-the-art (SOTA)

TABLE I
RELATED WORK

	Model	Backbone	Structure	Parameter	C1
Lightweight	PatchAD	MLP	Deep	Many	×
	DCdetector	Attention	Deep	Many	×
	ANNet	MLP+LSTM	Deep	Many	×
	LightDNN	CNN	Deep	Many	✓
	RG-GLD	GCN+KD	Deep	Many	×
	Optiforest	Isolation forest	Shallow	Few	×
	DIF	Isolation forest	Shallow	Few	×
	COUTA	OCSVM	Shallow	Few	×
	VQ-OCSVM	OCSVM	Shallow	Few	×
	FedPCA	PCA	Shallow	Few	×
privacy-preserving	FADngs	Density Estimation+KD	Deep	Many	×
	FedAnomaly	Transformer	Deep	Many	×
	BCFL	GAN	Deep	Many	✓
	Fed-SCR	GAN	Deep	Many	×
	OP-ADIHT	IHTable	Shallow	Few	×
	TRIDENT	Gaussian	Shallow	Few	✓

C1: Whether deployed directly on edge devices.

models. Notably, its detection time and model parameters are only one-tenth of those in competing models.

II. RELATED WORK

In this section, we respectively summarize the related work on lightweight anomaly detection and privacy-preserving anomaly detection, as shown in Table I.

A. Lightweight Anomaly Detection

To achieve a balance between high ACC, high timeliness, and low-resource consumption, existing lightweight models are generally classified into two categories.

The first category directly compresses deep neural network-based models with high ACC. For example, the patch-based anomaly detection (PatchAD) model [11] constructs a lightweight two-branch contrastive network using an Multi-Layer Perceptron (MLP)-based architecture inspired by the dual attention contrastive anomaly detection (DCdetector) model [8]. Similarly, the lightweight neural network-based anomaly detection (ANNet) model [12] optimizes Long Short Term Memory (LSTM) blocks with MLPs. The lightweight deep neural network-based anomaly detection (LightDNN) model [18] compresses the redundant parameters of the convolutional neural network (CNN), and tests the performance on edge devices. The reconstructed graph with global-local distillation (RG-GLD) model [19] integrates graph neural networks(GNN) and knowledge distillation (KD) to enhance timeliness and reduce resource consumption. Despite these advancements, compressed models retain deep structures with numerous parameters, which limits their potential for significant improvements in timeliness and resource efficiency.

The second category enhances the ACC of shallow machine learning or statistical models, which inherently offer high speed and low-resource consumption. For instance, the optimal isolation forest-based anomaly detection (Optiforest) model [13]

incorporates clustering learning and hashing to design an optimal multibranch tree structure. The DIF model [14] employs a customized shallow neural network to improve the ACC of isolation forest. The calibrated one-class classification-based anomaly detection (COUTA) model [15] designs multiple calibrated mechanisms for one-class support vector machine (OCSVM). Similarly, the VQ-OCSVM model [20] combines the vector quantization and OCSVM. The federated unsupervised anomaly detection (FedPCA) model [21] leverages principal component analysis (PCA), federal learning and alternating directions method multipliers to boost ACC. Although these models deliver superior timeliness and efficiency, their ACC often falters when applied to industrial time series with complex feature patterns.

B. Privacy-Preserving Anomaly Detection

To get privacy-preserving, most existing models utilize encryption, digital signatures, blockchain, and other protection techniques to enhance security. For example, the federated anomaly detection with noisy global density estimation (FADngs) model [17] uses a federated signature mechanism to guarantee identity authorization and data privacy. Similarly, the federated learning-based anomaly detection (FedAnomaly) model [22] develops a novel collaborative learning protocol to secure cloud-edge communications. The federated semisupervised class-rebalanced (Fed-SCR) model [23] applies the federal learning to preserve data privacy. Meanwhile, the blockchain-empowered anomaly detection (BCFL) model [24] adopts a blockchain-enabled decentralized asynchronous federated learning framework. In addition, the Shagufta et al. [16] designs a lightweight, aggregation-optimized encryption scheme. The OP-ADIHT model [25] uses interval hash table (ADIHT) and homomorphic encryption to construct a privacy-preserving anomaly detection scheme. However, these models often require encryption and decryption steps before performing anomaly detection. Consequently, they require longer processing times or consume more computational and storage resources. On resource-efficient IoT edge devices, some bulky privacy-preserving solutions may not run smoothly, which results in anomaly detection not working smoothly, or even not working at all.

C. Summary

Coming to Industry 4.0, detecting anomalies directly on industrial IoT edge devices is becoming increasingly critical. However, the resource limitations of edge devices necessitate the development of lightweight anomaly detection models with minimal parameters and low-resource consumption, all while maintaining high ACC. In addition, industrial IoT edge devices in industrial scenarios are often deployed in insecure environments. This requires anomaly detection models to protect data privacy without introducing significant additional overhead.

III. PROPOSED MODEL

A. Problem Statement

Suppose M sensors are employed to observe a dynamic production system over T times. The collected data is multisensor temporal data, also known as a multivariate time series. Multisensor temporal data consists of both temporal and variable

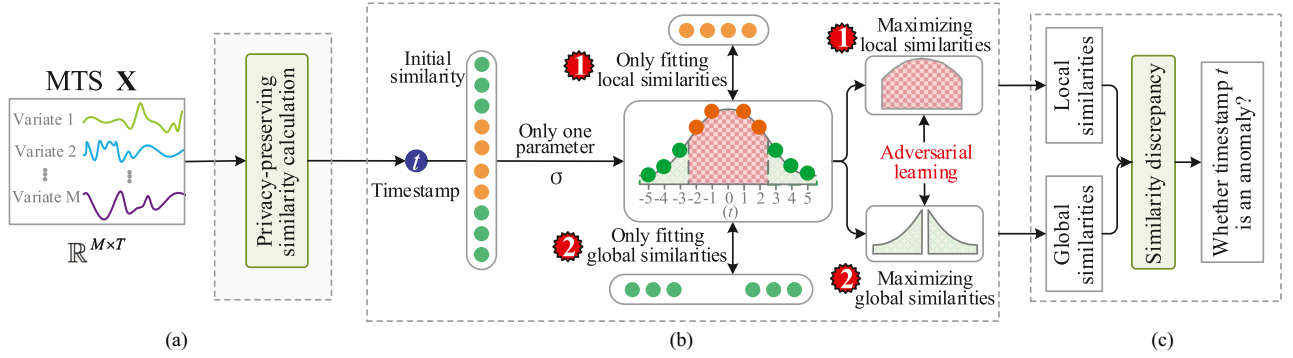


Fig. 2. Overview of PPLAD model. (a) Initial similarity calculation. (b) Tiny trainable Gaussian distribution-based adversarial learning. (c) Global and local similarity discrepancy-based anomaly scoring.

(spatial) dimensions. In the temporal dimension, the multisensor temporal data can be formulated as $\mathbf{X} = \{x_t \in \mathbb{R}^{M \times 1}\}_{t=1}^T$, where x_t represents the observed value of M variables at the t th timestamp. In the variable dimension, the multisensor temporal data can be formulated as $\mathbf{X} = \{x^m \in \mathbb{R}^{1 \times T}\}_{m=1}^M$, where x^m represents the observed values from T timestamps at the m th variable.

Problem statement: This article aims to develop a lightweight anomaly detection model. *First*, unlabeled historical multisensor temporal data is used to train the anomaly detection model. *Second*, the trained model is used to generate an anomaly score for each to-be-tested timestamp

$$\text{Score}_t = \text{PPLAD}(x_t) \quad (1)$$

where $x_t \in \mathbb{R}^M$ denotes the observed values of M variables at the t th to-be-tested timestamp. *Finally*, the anomaly score determines whether a timestamp is classified as an anomaly

$$\bar{y}_t = \begin{cases} 1, & \text{if } \text{Score}_t \geq \alpha \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

where α is a preset threshold ranging from 0 to 1. If $\bar{y}_t = 1$, the t th to-be-tested timestamp is labeled as an anomaly; otherwise, it is classified as normal.

B. Overview

To detect anomalies directly on the resource-limited edge devices, a trainable Gaussian distributions-based privacy-preserving lightweight anomaly detection model, named PPLAD, is proposed via global and local similarity discrepancy. To the best of authors' knowledge, this is the first work to build an industrial anomaly detection model using only learnable Gaussian distributions. The overview of PPLAD is illustrated as Fig. 2, which consists of the following three components.

- 1) *Initial similarity calculation:* This component calls a third-party security interface to generate the initial similarities of each timestamp with its local and global neighbors as input for the model.
- 2) *Gaussian distribution-based adversarial learning:* In this component, a trainable Gaussian distribution with only one parameter is constructed for each timestamp, so as to model its global and local similarities based on initial similarities. Meanwhile, an adversarial learning mechanism

is designed to amplify the discrepancy between global and local similarities.

- 3) *Anomaly scoring:* This component exploits the discrepancy between global and local similarities to determine if a timestamp is an anomaly.

C. Initial Similarity Calculation

Unlike traditional models that directly use raw private data, PPLAD utilizes data similarities obtained from third-party secure interfaces as model input to ensure privacy preservation. These interfaces can be located on edge devices or in the cloud. In this case, data privacy is not only preserved on insecure edge devices, but also the coupling between multiple sensors can be captured.

In this section, a similarity calculation strategy is designed to serve as a recommended solution for third-party references. *First*, for each timestamp, the M variables in the raw data $\mathbf{X} \in \mathbb{R}^{M \times T}$ form a data vector $x_t \in \mathbb{R}^{M \times 1}$. *Second*, a similarity technique—such as distance similarity (Mean Squared Error, MSE), probability distribution similarity (Kullback-Leibler divergence, KL), or vector similarity (Cosine)—is used to compute the similarity between data vectors of any two timestamps. *Finally*, the raw data is transformed into a similarity matrix $\mathbf{S} \in \mathbb{R}^{T \times T}$, where $s_t \in \mathbb{R}^{T \times 1}$ represents the similarities between timestamp t and its T neighbors.

D. Gaussian Distribution-Based Adversarial Learning

Industrial sensor signals often show periodicity, seasonality, and trends. This suggests that a normal timestamp is not only related to neighbor timestamps (local neighbors), but also to distant timestamps (global neighbors). Conversely, outliers typically persist across multiple timestamps and show significant deviations from normal values. This suggests that an abnormal timestamp is only related to adjacent timestamps (local neighbors). Therefore, the discrepancy between global and local similarities for each timestamp is a useful metric for identifying anomalies. Based on this insight, a trainable Gaussian distribution with only one parameter is constructed for each timestamp to learn global and local similarities from the initial similarities, as shown in Fig. 2(b). To enhance this process, a customized adversarial learning mechanism is designed to amplify the discrepancy between global and local similarities. In adversarial learning mechanism, local and global similarity learning are treated as competing processes vying for the same Gaussian distribution.

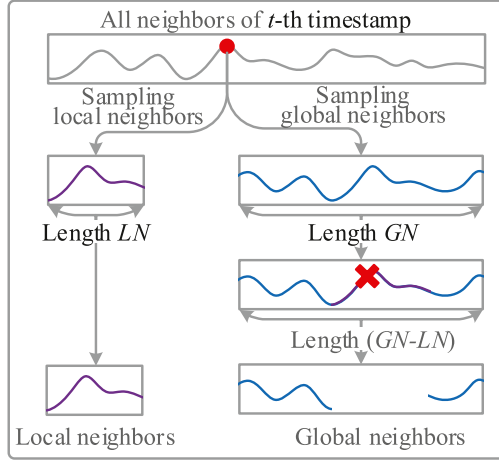


Fig. 3. Global and local sampling.

Local similarity learning wants to monopolize the Gaussian distribution itself. Specifically, the Gaussian distribution for a timestamp should perfectly fit the initial local similarities, and the area occupied by the local neighbors in the Gaussian distribution should be as large as possible. On the contrary, global similarity learning also wants to monopolize the Gaussian distribution itself. The Gaussian distribution is expected to fit only the initial global similarities, and the area occupied by the global neighbors is as large as possible. Through this adversarial learning, the Gaussian distribution adapts to fit both local and global similarities simultaneously for normal timestamps. However, for abnormal timestamps, such simultaneous fitting becomes challenging. As a result, the discrepancy between global and local similarities diminishes for normal timestamps but increases for abnormal ones, enabling effective anomaly detection.

Taking the t th timestamp as an example, the adversarial learning process is stated as follows.

1) **Global and Local Neighbor Sampling:** Timestamps within the range $[t - \frac{LN}{2}, t + \frac{LN}{2}]$ are selected as local neighbors for timestamp t . Meanwhile, timestamps with the ranges $[t - \frac{GN}{2}, t - \frac{LN}{2}]$ and $[t + \frac{LN}{2}, t + \frac{GN}{2}]$ are chosen as global neighbors. The sampling process is shown in Fig. 3. Based on the coordinates of local and global neighbors, the corresponding initial similarities are extracted from $s_t \in \mathbb{R}^{T \times 1}$ and normalized as follows:

$$\bar{s}_{it} = \frac{1}{1 + \exp^{-(s_{it})}} \quad (3)$$

where s_{it} is initial similarity between timestamps i and t .

2) **Trainable Gaussian Distribution:** A private trainable Gaussian distribution with only one parameter is constructed for timestamp t , and expressed as follows:

$$g_t(x) = \frac{1}{\sqrt{2\pi}\sigma_t} \exp\left[-\frac{(x-t)^2}{2\sigma_t^2}\right] \quad (4)$$

where x indicates the coordinate of a neighbor. σ_t indicates the variance of this distribution, it is a trainable parameter and its value is initialized by a common two-layer MLP network.

3) **Similarity Learning for Local Neighbors:** First, the variance σ_t is trained to make the Gaussian distribution only fit the normalized similarity of each local neighbor as closely as

possible

$$L_1 = \frac{1}{LN} \sum_{i=t-\frac{LN}{2}}^{t+\frac{LN}{2}} [g_t(i) - \bar{s}_{it}]^2 \quad (5)$$

where LN indicates the number of local neighbors and L_1 is the first train loss function.

Second, the area of local similarities is expected to be maximized within the overall distribution

$$L_2 = 1 - \left[\text{Area}\left(t + \frac{LN}{2}\right) - \text{Area}\left(t - \frac{LN}{2}\right) \right] \quad (6)$$

where $\text{Area}(x)$ represents the area of the Gaussian distribution in the range $[-\infty, x]$, L_2 is the second train loss function.

4) **Similarity Learning for Global Neighbors:** First, the Gaussian distribution is expected to only fit normalized similarity of each global neighbor as closely as possible

$$L_3 = \frac{1}{GN - LN} \left[\sum_{i=t-\frac{GN}{2}}^{t-\frac{LN}{2}} (g_t(i) - \bar{s}_{it})^2 + \sum_{j=t+\frac{LN}{2}}^{t+\frac{GN}{2}} (g_t(j) - \bar{s}_{jt})^2 \right] \quad (7)$$

where GN indicates the number of global neighbors and L_3 is the third loss function.

Second, the area of global similarities is expected to be maximized within the overall distribution

$$L_4 = 1 - 2 \times \text{Area}\left(t - \frac{LN}{2}\right) \quad (8)$$

where L_4 is the fourth loss function.

5) **Adversarial Learning:** In the abovementioned process, local similarity learning and global similarity learning constitute two competitors who play against each other. Therefore, the final loss function for adversarial learning is the sum of the local and global losses

$$\text{Loss} = L_1 + L_2 + L_3 + L_4. \quad (9)$$

E. Anomaly Scoring

After the PPLAD model is trained, a test timestamp t first uses the PPLAD model to generate its Gaussian distribution. Then, the discrepancy between global and local similarities serves as the final anomaly score for timestamp t

$$\text{Score}_t = |L_1 - L_3|. \quad (10)$$

where $|\cdot|$ means absolute value, L_1 represents the Gaussian-fit-error for the local similarities of timestamp t , and L_3 indicates the Gaussian-fit-error for the global similarities of timestamp t . Finally, (2) is employed to determine whether timestamp t is an anomaly.

F. Algorithm and Analysis

The pseudocode for PPLAD is shown in Algorithm 1.

Complexity: The PPLAD model consists of four stages: initial similarity calculation, Gaussian distribution-based adversarial learning, anomaly scoring, and anomaly detection. In the first stage, the initial similarity calculation is handled by a third-party library, and its complexity is, therefore, omitted. In the second stage, a private, trainable Gaussian distribution is created

Algorithm 1: PPLAD.

input : Time-series data $\mathbf{X} \in \mathbb{R}^{M \times T}$; sampling parameters LN and GN ; parameter α .
output: Anomaly labels for all timestamps $\bar{\mathbf{y}}$.

// Initial similarity calculation
1 $\mathbf{S} \in \mathbb{R}^{T \times T} \leftarrow$ Calling a third-party library to generate initial similarities of $\mathbf{X} \in \mathbb{R}^{M \times T}$;
2 $\bar{\mathbf{S}} \in \mathbb{R}^{T \times T} \leftarrow$ Similarity normalization for $\mathbf{S}$;
// Gaussian adversarial Learning
3 **for** $t \in \{1, \dots, T\}$ **do**
 // Trainable Gaussian Construction
4 $g_t(x) = \frac{1}{\sqrt{2\pi}\sigma_t} \exp\left[-\frac{(x-t)^2}{2\sigma_t^2}\right]$ for t ;
 // Sampling
5 Sampling local neighbors of length LN and global neighbors of length $GN - LN$;
 // Maximizing local similarities
6 **for** $i \in \{1, \dots, LN\}$ **do**
7 $g_t(i) = \bar{s}_{it} \leftarrow$ Fitting similarity;
8 **end**
9 $\sum \{g_t(i)\}_{i=1}^{LN} = 1 \leftarrow$ Maximize area;
 // Maximizing global similarities
10 **for** $j \in \{1, \dots, (GN - LN)\}$ **do**
11 $g_t(j) = \bar{s}_{jt} \leftarrow$ Fitting similarity;
12 **end**
13 $\sum \{g_t(j)\}_{j=1}^{(GN-LN)} = 1 \leftarrow$ Maximize area;
14 **end**
 // Anomaly Scoring
15 **for** $t \in \{1, \dots, T\}$ **do**
16 $Score_t \leftarrow$ Local-global fit discrepancy by Eq.10;
17 **end**
 // Anomaly detection
18 $\bar{y}_t = \begin{cases} 1, & \text{if } Score_t \geq \alpha \\ 0, & \text{otherwise} \end{cases}$
19 **Return** $\bar{\mathbf{y}}$;

for each timestamp. Adversarial learning is then applied to this distribution by maximizing both local and global similarities. Specifically, the complexity of the sampling process is $\mathcal{O}(T \times GN)$, the complexity of the Gaussian Construction is $\mathcal{O}(T)$, the complexity of the local similarity learning is $\mathcal{O}(T \times LN)$, and the complexity of the global similarity learning is $\mathcal{O}(T \times (GN-LN))$. Consequently, the overall complexity for this stage is $\mathcal{O}(T \times (2 \times GN + 1))$, where $GN \ll T$. In the third stage, the discrepancy between local and global fitting is computed as the anomaly score for each timestamp. Its complexity is $\mathcal{O}(T)$. In the final stage, each timestamp is determined whether to be an anomaly, its complexity is $\mathcal{O}(T)$. In summary, the overall time complexity of the PPLAD model is $\mathcal{O}(T \times GN)$, making it linear and nearly linear given that $GN \ll T$.

IV. EXPERIMENT**A. Experimental Goal**

We conduct extensive experiments to answer the following four research questions.

TABLE II
DATASET

Dataset	Features	Train set	Test set	URL
HAI	86	280 800	54 000	HAI-URL
MSL	55	58 317	73 729	MSL-URL
PSM	25	132 481	87 841	PSM-URL
SKAB	8	12 450	5710	SKAB-URL
SMAP	25	135 183	427 617	SMAP-URL
SMD	38	708 405	708 420	SMD-URL
PUMP	51	132 192	88 128	PUMP-URL
SWaT	51	495 000	449 919	SWaT-URL
WaDi	127	103 680	69 121	WaDi-URL

- 1) *RQ-1 (ACC)*: Does our model outperform baseline models in detection ACC on multisensor temporal datasets?
- 2) *RQ-2 (Deployability)*: Is our model easily deployable on resource-constrained industrial IoT edge devices?
- 3) *RQ-3 (Timeliness and resource consumption)*: How does our model perform in terms of timeliness and resource consumption on industrial IoT edge devices?
- 4) *RQ-4 (Ablation)*: Does each innovative component of our model contribute to overall performance?
- 5) *RQ-5 (Case Study)*: Can PPLAD perform well in real industrial environments?

B. Dataset

The following nine publicly available real-world industrial multisensor temporal datasets are selected for our experiments, as shown in Table II.

- 1) *HAI* (HIL-based Augmented ICS Dataset): Multiple industrial signals from pumps, valves, pressure sensors, and other common industrial equipment.
- 2) *MSL* (Mars Science Laboratory Dataset): Operating status of multiple sensors or controllers on the Mars rover.
- 3) *PSM* (Pooled Server Metrics Dataset): 25-dimensional monitoring information from eBay server machines.
- 4) *PUMP* (Pump Sensor Dataset): Inspection data from multiple sensors on pumps.
- 5) *SKAB* (Skoltech Anomaly Benchmark Dataset): Multisensor temporal signals obtained from industrial test benches.
- 6) *SMAP* (Soil Moisture Active Passive Satellite Dataset): NASA's SMAP satellite signals that record moisture levels at varying soil depths.
- 7) *SMD* (Server Machine Dataset): Operational status from multiple servers.
- 8) *SWaT* (Secure Water Treatment (SWaT) Dataset): 51-dimensional data collected by multiple sensors in public water treatment infrastructure.
- 9) *WaDi* (Water Distribution Dataset): 127-dimensional monitoring data from an industrial control system.

C. Baseline, Metric, Environment

1) *Baseline*: Eight SOTA models are selected as the baselines, as listed in Table III. Among these, PatchAD, NPSR, DTAAD, DIF, TFMAE, and DCdetector are deep models with numerous parameters, while PPLAD, COUTA, and Optiforest are lightweight models with fewer parameters.

TABLE III
BASELINE

Models	Full name	Year	Source Code
COUTA [15]	Calibrated One-class Classification for Unsupervised Time Series Anomaly Detection	2024	github.com/xuhongzuo/COUTA
PatchAD [11]	PatchAD: A Lightweight Patch-Based MLP-Mixer for Time Series Anomaly Detection	2024	github.com/EmorZz1G/PatchAD
NPSR [7]	Nominality score conditioned time series anomaly detection by point/sequential reconstruction	2024	github.com/andrewlai61616/NPSR
DTAAD [6]	DTAAD: Dual TCN-Attention Networks for Anomaly Detection in Multivariate Time Series Data	2024	github.com/Yu-Lingrui/DTAAD
DIF [14]	Deep isolation forest for anomaly detection	2023	github.com/xuhongzuo/deep-iforest
DCdetector [8]	DCdetector: Dual Attention Contrastive Representation Learning for Time Series Anomaly Detection	2023	github.com/DAMO-DI-ML/KDD2023-DCdetector
Optiforest [13]	OptIForest: Optimal Isolation Forest for Anomaly Detection	2023	github.com/xiaogl/OptIForest
TFMAE [26]	Temporal-Frequency Masked Autoencoders for Time Series Anomaly Detection	2024	github.com/LMissher/TFMAE

2) **Metric:** To ensure a comprehensive evaluation, four widely used metrics are employed: ACC, precision, recall, and $F1$ -Score ($F1$).

- 1) **ACC** measures the proportion of correctly predicted samples relative to the total number of samples.
- 2) **Precision** indicates the proportion of accurately predicted normal timestamps among all predicted normal timestamps.
- 3) **Recall** reflects the proportion of actual normal timestamps that are correctly identified.
- 4) **$F1$** provides a balanced evaluation by combining *recall* and *precision*.

In general, these metrics range from 0 to 1, with higher values indicating better anomaly detection performance.

3) **Environment:** The primary experiments are conducted on a Windows 11 system equipped with an Intel Core i7-10700KF CPU, an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), and 64 GB of RAM. Deployment experiments are performed on two resource-constrained IoT edge devices: Raspberry Pi 4b and NVIDIA Jetson Xavier NX. The PPLAD model is implemented in Python using PyCharm as the integrated development environment. For the other models, official Python implementations are obtained from the websites of respective authors, as detailed in Table III.

D. RQ-1 (ACC)

To answer RQ-1, the PPLAD model is compared against seven baselines across eight multisensor datasets.

1) **Accuracy:** Table IV presents the detection ACC of the eight models across eight datasets. Bold text in the table denotes the highest performance, while underlined text indicates the second best. Several observations can be made from the table.

- 1) **Overall performance:** PPLAD achieves the best performance, followed by DCdetector, COUTA, PatchAD, DIF, and DTAAD, while Optiforest and NPSR show the weakest performance.
- 2) **Stability:** PPLAD demonstrates the most stable performance across all four metrics, with DCdetector, DIF, DIF following closely. Specifically, PPLAD achieves first or second rankings across all datasets and all metrics.
- 3) **Deep versus shallow models:** Deep neural network-based models (DIF, DCdetector, DTAAD, PatchAD) generally outperform shallow models (Optiforest, COUTA).

However, PPLAD, despite being a shallow model, surpasses several deep models. For instance, in the MSL dataset, PPLAD

achieves an ACC of 0.9919 and a recall of 0.9885, compared to DTAAD's ACC of 0.9242 and recall of 0.3754, and DCdetector's ACC of 0.9883 and recall of 0.9703.

2) **Visualization:** We visualize anomaly detection results of nine models for timestamps 60 500 to 63 500 in the 1th variable on the MSL dataset, as illustrated in Fig. 4. In this figure, the black curve represents the real data, while the red curve corresponds to the predicted labels (peaks indicate outliers and valleys denote normal values). Two key observations can be drawn from Fig. 4. **ACC:** PPLAD, TFMAE, COUTA, and DCdetector exhibit higher ACC compared to other models. For instance, within the timestamps of 62 300 to 62 600, PPLAD, TFMAE, COUTA, and DCdetector detection results closely align with the real labels, while other models exhibit greater errors. **False alarm:** PPLAD, COUTA, and DCdetector demonstrate fewer false alarms than other models.

3) **Conclusion:** The experimental results show that the PPLAD model achieves superior ACC and stability in anomaly detection on industrial multisensor datasets.

E. RQ-2 (Deployability)

To address RQ-2, deployment experiments are conducted for nine models on two edge devices: the Raspberry Pi 4b with little resources and the Jetson Xavier NX with normal resources. Specifically, the Raspberry Pi 4b is equipped with a 1.5 GHz ARM Cortex-A72 processor and 2 GB of RAM, while the Jetson Xavier NX has a 6-core Carmel ARMv8.2 processor with 8 GB of RAM. In these experiments, deep models are initially trained in the cloud before being deployed on edge devices for testing. Meanwhile, the shallow models are deployed on the edge devices directly. The deployment test results are illustrated in Fig. 5.

From the figure, two key observations can be made. **Deployability:** All models except the PatchAD are successfully deployed on the edge devices, with lightweight models demonstrating greater ease of deployment. **Convenience:** In industrial IoT environments, frequent changes in production processes cause dynamic fluctuations in time signals. As a result, we need to update the model regularly and keep its best performance. For deep models, the update process requires fine-tuning in the cloud with new data samples and then redeploying the updated model to the edge devices. In contrast, lightweight models can be updated directly on edge devices, consuming fewer data samples and reducing communication overhead. **Summary:** Shallow lightweight models are more easily deployable on resource-constrained Industrial IoT edge devices than their deeper counterparts.

TABLE IV
ACCURACY ANALYSIS

		PPLAD	COUTA	PatchAD	NPSR	DTAAD	DIF	DCdetector	Optiforest	TFMAE
HAI	ACC	0.9909	0.9662	0.9652	0.9715	0.9652	0.4693	0.9745	0.8921	0.9864
	Precision	<u>0.8578</u>	0.7673	<u>0.8274</u>	0.7473	<u>0.8274</u>	0.6377	0.6845	0.5784	0.9499
	Recall	1	0.5565	0.468	0.7133	0.468	0.5646	1	0.829	0.7957
	F1	0.9235	0.6451	0.5978	0.7299	0.5978	0.5989	0.8127	0.6724	0.866
MSL	ACC	0.9919	0.9757	0.9652	0.9627	0.9242	0.5289	0.9883	0.6731	0.9906
	Precision	<u>0.9376</u>	0.8983	0.9477	0.8235	0.8005	0.8754	0.9228	0.1392	0.9332
	Recall	<u>0.9885</u>	0.8734	0.7096	0.8137	0.3754	0.7663	0.9703	0.4059	0.9932
	F1	0.9624	0.8857	0.8114	0.8185	0.5111	0.8172	0.946	0.2073	0.9569
PSM	ACC	0.9904	0.913	0.9394	0.8324	0.925	0.6364	0.9885	0.3088	0.9855
	Precision	<u>0.9844</u>	0.9441	0.884	0.6308	0.8412	0.9792	0.9761	0.2833	0.9935
	Recall	<u>0.981</u>	0.7303	0.9011	0.9241	0.9075	0.9486	0.9827	0.9742	0.9541
	F1	0.9827	0.8235	0.8926	0.7498	0.8732	0.9199	0.9794	0.4389	0.9734
PUMP	ACC	0.9985	0.9836	0.9957	0.9889	0.9166	0.8645	0.9961	0.8211	0.9984
	Precision	<u>0.9805</u>	0.7572	0.931	0.8649	0.8079	0.9653	0.9407	0.2124	0.9839
	Recall	<u>0.9901</u>	0.9903	0.99	0.9153	0.8638	0.9904	0.9729	0.9667	0.9834
	F1	0.9853	0.8581	0.9595	0.8893	0.8936	0.9777	0.9606	0.3483	0.9837
SKAB	ACC	0.9963	0.912	0.9286	0.8864	0.9776	0.4186	0.9938	0.3503	0.9961
	Precision	<u>0.9896</u>	0.9644	0.9967	0.9333	0.9405	0.9804	0.9828	0.3503	0.9891
	Recall	1	0.7773	0.8004	0.7345	1	<u>0.9901</u>	1	1	1
	F1	0.9948	0.8609	0.8876	0.822	0.9693	0.9848	0.9913	0.5188	0.9945
SMAP	ACC	0.9909	0.9418	0.9395	0.9666	0.9365	0.4206	0.9843	0.2571	0.9903
	Precision	<u>0.9433</u>	0.9717	0.9216	0.8541	0.8981	0.7346	0.9064	0.1363	0.9375
	Recall	<u>0.9885</u>	0.5614	0.576	0.8907	0.5694	0.8177	0.9786	0.9008	0.9905
	F1	0.9654	0.7117	0.7089	0.8721	0.6969	0.9472	0.9411	0.2367	0.9633
SMD	ACC	0.9877	0.9757	0.9661	0.9786	0.9773	0.922	0.9877	0.9506	0.9876
	Precision	<u>0.8151</u>	0.7022	0.5554	0.7415	0.7053	0.539	<u>0.8129</u>	0.4559	0.8093
	Recall	<u>0.9121</u>	0.7197	0.9212	0.7461	0.7789	0.4361	0.9133	0.3103	<u>0.918</u>
	F1	0.8609	0.7108	0.693	0.7438	0.7403	0.4823	<u>0.8602</u>	0.4791	<u>0.8602</u>
SWaT	ACC	0.9956	0.9627	0.9656	0.947	0.7711	0.9466	0.9913	0.3117	0.9945
	Precision	<u>0.965</u>	0.926	0.984	0.8174	0.3232	0.9477	0.9331	0.1395	0.9636
	Recall	1	0.7552	0.7301	0.7255	0.8084	0.8036	1	0.9032	0.9923
	F1	0.9822	0.832	0.8382	0.7687	0.4618	0.8697	0.9654	0.2416	0.9777
WaDi	ACC	0.9943	0.9861	0.9793	0.9757	0.9531	0.7512	0.9915	0.6488	0.994
	Precision	0.8943	0.8093	0.83	0.7401	0.5354	0.6348	0.851	0.0751	0.8898
	Recall	1	<u>0.9392</u>	0.7238	0.7659	0.2494	0.801	1	0.5533	1
	F1	0.9442	0.8691	0.7732	0.7528	0.3403	0.7083	<u>0.9195</u>	0.1322	0.9417

The bold text denotes the highest performance, while underlined text indicates the second best.

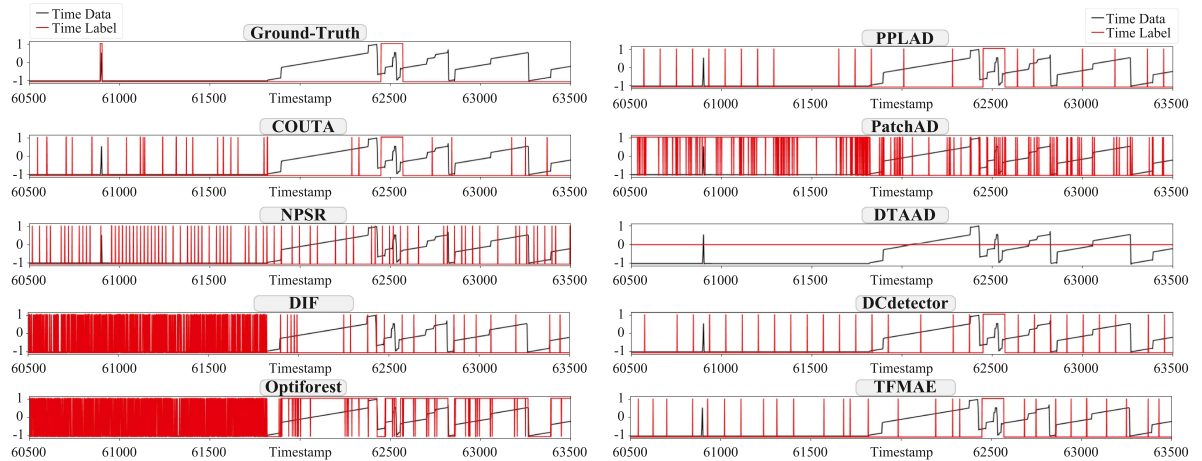


Fig. 4. Visualization of anomaly detection at timestamps 60500–63500 of 1th-variable on MSL dataset.

F. RQ-3 (Timeliness and Resource Consumption)

To answer RQ-3, the timeliness and resource consumption of eight models are evaluated using the MSL dataset across three deployment environments: a thick PC, a thin Jetson Xavier NX, and a micro Raspberry Pi 4b. Table V lists the timeliness and resource consumption of eight models, where “Or” denotes out-of-memory. From the table, several key findings can be observed. *Model parameters*: The PPLAD model contains only

12.0K parameters, significantly fewer than other models such as COUTA (376.1K), PatchAD (2072.9 K), and DCdetector (826.5 K). *Timeliness*: For training one epoch with a batch size of 128, the PPLAD needs just 10.3 s, whereas PatchAD takes 229.9 s, DCdetector requires 128.1 s, and COUTA takes 26.5 s. Similarly, PPLAD completes testing all timestamps in 0.9 s, outperforming PatchAD (154.5 s), COUTA (3.7 s), and DCdetector (2.9 s). *Computing resource consumption*: Shallow models show substantially lower CPU and GPU usage compared

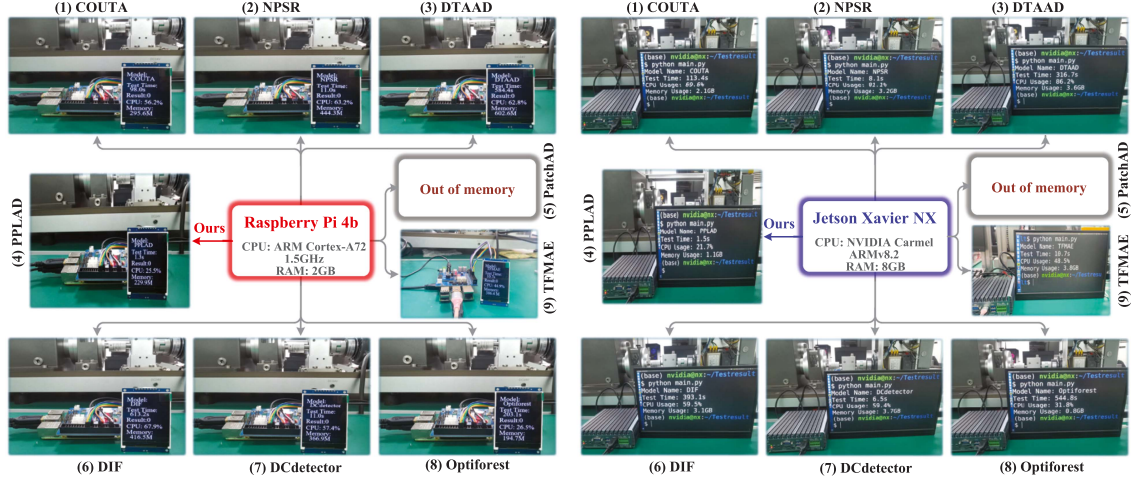


Fig. 5. Deployment results on two industrial IoT edge devices, where Raspberry Pi 4b on the left, Jetson Xavier NX on the right.

TABLE V
TIMELINESS AND RESOURCE CONSUMPTION

		PPLAD	COUTA	PatchAD	NPSR	DTAAD	DIF	DCdetector	Optiforest	TFMAE
PC with powerful resources (CPU: i7-10700KF GPU: RTX 3090 RAM: 64G VGA-RAM: 24G)	Model parameters	12.0K	376.1K	2072.9K	207.1K	123.3K	-	826.5K	-	706.0K
	One-epoch-train-time	10.3s	26.5s	229.9s	30.4s	41.9s	246.7s	128.1s	81.7s	26.5s
	All-timestamps-test-time	0.9s	3.7s	154.5s	11.0s	62.5s	219.6s	2.9s	388.8s	24.1s
	CPU Usage	8.0%	44.3%	15.1%	35.5%	7.4%	25.1%	13.5%	32.4%	11.0%
	GPU Usage	14.1%	29.6%	80.9%	24.1%	83.5%	20.9%	83.9%	-	16.0%
	RAM Usage	2.2GB	5.3GB	13.5GB	3.1GB	7.2GB	8.8GB	4.2GB	1.1GB	4.8GB
	VGA-RAM Usage	0.8GB	0.9GB	11.1GB	2.6GB	6.4GB	1.7GB	8.1GB	-	4.4GB
Jetson Xavier NX (CPU: Carmel ARMv8.2 RAM: 8G)	All-timestamps-test-time	1.5s	113.4s	Ot	8.1s	316.7	393.1s	6.5s	544.8s	7.4s
	CPU Usage	21.7%	69.6%	Ot	81.1%	86.2%	59.5%	59.4%	31.8%	48.5%
	RAM Usage	1.1GB	2.1GB	Ot	3.2GB	3.6GB	3.1GB	3.7GB	0.8GB	3.8GB
Raspberry Pi 4b (CPU: ARM Cortex-A72 RAM: 2G)	10000-timestamps-test-time	1.3s	98.0s	Ot	11.0s	584.4s	613.2s	11.0s	203.1s	10.7s
	CPU Usage	25.5%	56.2%	Ot	63.2%	62.8%	67.9%	57.4%	26.5%	44.9%
	RAM Usage	229.9MB	295.6MB	Ot	444.3MB	602.6MB	416.5MB	366.9MB	194.7MB	386.4MB

to deep models. For example, on the Raspberry Pi 4b, CPU usage for PPLAD is 25.5%, while it is 57.4% for DCdetector and 63.2% for NPSR. *Storage resource consumption:* Deep models require substantially more storage. For example, on the Jetson Xavier NX environment with only 8 G RAM, the RAM Usage is 3.1 GB for deep DIF, 3.7 GB for deep DCdetector, while only 1.1 GB for our PPLAD, and 0.8 GB for shallow Optiforest. *In summary:* PPLAD demonstrates strong timeliness and low-resource consumption, making it highly suitable for resource-constrained industrial IoT edge devices.

G. RQ-4 (Ablation)

To address RQ-4, two ablation experiments are conducted to evaluate key components of the PPLAD model. These experiments employ four multisensor datasets (HAI, PUMP, SMAP, and SWaT), and three metrics (ACC, Recall, and $F1$).

1) *Input Similarity Calculation:* The first key component is to utilize data similarity from the third-party interfaces or other models to replace the raw private data and thus achieve data privacy protection. Therefore, this experiment assesses the impact of different initial similarity calculation methods on model performance. Specifically, we design four contrasting models: MSE-based distance similarity, Cosine-based vector similarity, KL-based probability distribution similarity, and attention-based

association similarity. Fig. 6(a) shows the detection performance of four models across four datasets. From the figure, we can find that the performance of the four models is very similar across the four datasets. It suggests that the PPLAD model is robust to different initial similarity calculation methods. For example, on the SMAP dataset, the ACC scores are 0.9909 for the MSE-based model, 0.9901 for the Cosine-based model, 0.9903 for the KL-based model, and 0.9892 for the attention-based model.

Table VI further lists the integration results of PPLAD with other models. From the table, the PPLAD model demonstrates flexibility in integrating with other models and improving their anomaly detection performance. For example, the trained DCdetector model is used to compute the initial neighbor similarities for each timestamp. Then, these similarities are optimized by the PPLAD model to better identify anomalies. Specifically, on the SMAP dataset, the ACC score improves from 0.9843 with the DCdetector model to 0.9892 with the integrated DCdetector+PPLAD model.

2) *Adversarial Learning:* The second key component is an adversarial learning mechanism designed to amplify the discrepancy between local similarity and global similarity. In the adversarial learning mechanism, there are two adversarial parts: similarity adversarial learning, and area adversarial learning. Therefore, this experiment evaluates the effect of two adversarial parts. Specifically, we design three competition models: only

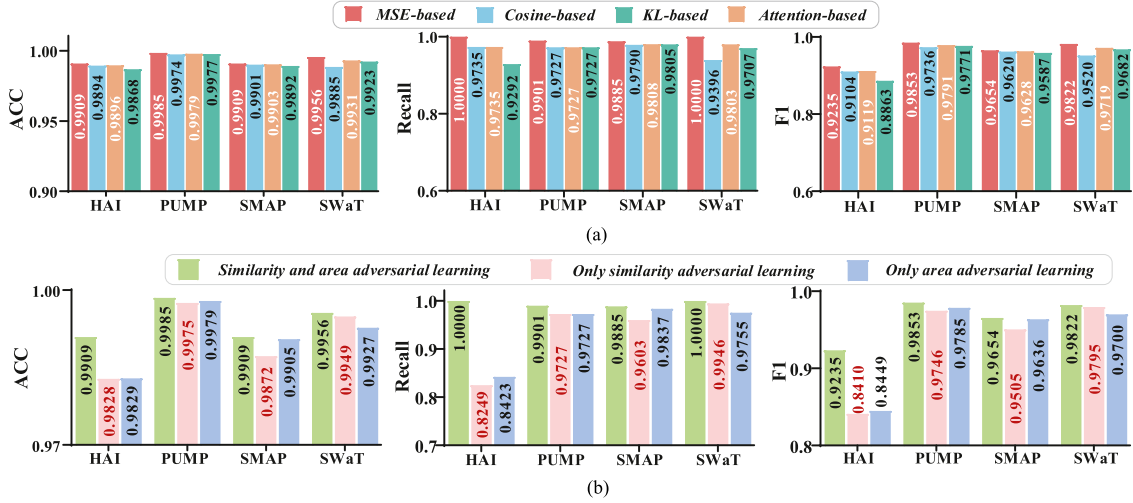


Fig. 6. Two ablation experiment results. (a) Performance analysis for different initial similarity. (b) Performance analysis for adversarial learning.

TABLE VI
INTEGRATION WITH OTHER MODELS

		DCdetector	DCdetector+Our	DIF	DIF+Our
HAI	ACC	0.9745	0.9868	0.4693	0.9879
	Recall	1	0.9992	0.5646	0.9457
	F1	0.8127	0.8863	0.5989	0.8965
PUMP	ACC	0.9961	0.9977	0.8645	0.9977
	Recall	0.9407	0.9727	0.9904	0.9727
	F1	0.9606	0.9771	0.9777	0.9769
SMAP	ACC	0.9843	0.9892	0.4206	0.9899
	Recall	0.9786	0.9805	0.8177	0.9766
	F1	0.9411	0.9587	0.9472	0.9613
SWaT	ACC	0.9913	0.9923	0.9466	0.9936
	Recall	1	0.9907	0.8036	0.9894
	F1	0.9654	0.9682	0.8697	0.9739

similarity adversarial learning (first model), only area adversarial learning (second model), and similarity and area adversarial learning (third model). Fig. 6(b) displays the detection ACC of the three models. From the figure, two observations can be obtained. *Overall performance*: The model combining similarity and area adversarial learning (third model) outperforms the other models across all metrics and datasets, as indicated by the highest bars in the figure. *Comparison of the first and second models*: Each of these models has strengths in different contexts. For instance, on the SWaT dataset, the first model achieves higher scores for ACC, recall, and F1. However, on the SMAP dataset, the second model performs better across all three metrics. *In summary*: The combination of similarity and area adversarial learning yields the best results.

H. RQ-5 (Case Study)

To answer RQ-5, a drive system abnormality and fault simulation testbed is employed to perform this experiment, its process is shown in Fig. 7.

1) *Physical Testbed*: The drive system abnormality and fault simulation testbed comprises a motor, a gearbox, a magnetic powder brake, a motor controller, an edge data collector, and control software. The testbed integrates a torque sensor, a three-axis

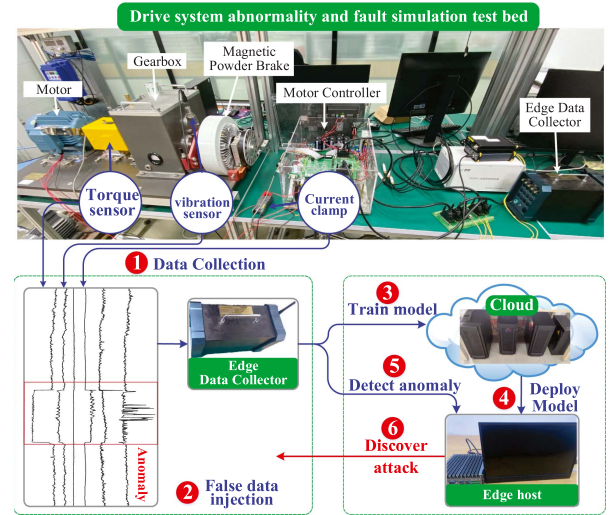


Fig. 7. Physical testbed and anomaly detection process.

vibration acceleration sensor, and a current and voltage clamp to collect signals for current, voltage, vibration, and torque.

2) *Data Collection and Anomaly Generation*: Based on this testbed, signal data were collected over 60 s at a frequency of 12.8 kHz. The collected data contains 6 variables: vibration signals along the x , y , and z axes, a torque signal, and current and voltage signals. With the aid of control software, false data injection attacks were applied to tamper with the real collected data. Specifically, 3 s of data were tampered with every 12 s, repeated 4 times. Based on the abovementioned process, two datasets with anomalies are generated. The first, a single-anomaly dataset, contains only the tooth-break-failure anomaly. The second, a mixed-anomaly dataset, includes both tooth-break-failure and outer-ring-failure anomalies. Each dataset contains 768 000 timestamps, with 537 600 used for training and the remainder for testing.

3) *Experiment Results*: Table VII summarizes the performance of all models on two datasets. *Focusing on the single*

TABLE VII
RESULTS OF ANOMALY DETECTION

Fault type	Metric	PPLAD	COUTA	PatchAD	NPSR	DTAAD	DIF	DCdetector	Optiforest	TFMAE
single anomaly	ACC	0.9961	0.9581	0.9641	0.9431	0.8882	0.2706	0.9872	0.7598	<u>0.9897</u>
	Precision	0.9786	0.8059	0.8303	0.7697	0.6089	0.1927	0.932	0.5962	<u>0.9448</u>
	Recall	1	1	0.8177	0.8093	0.7345	0.4206	<u>0.9786</u>	0.8153	1
	F1	0.9892	0.8925	0.9073	0.8698	0.7569	0.3231	0.9648	0.6888	<u>0.9716</u>
	Test time	2.1s	84.6s	Ot	8.6s	384.5s	535.4s	6.4s	516.5s	8.1s
Mixed anomaly	ACC	0.9929	0.956	0.8507	0.9773	0.7855	0.5554	<u>0.9809</u>	0.8644	0.9797
	Precision	<u>0.9685</u>	0.9897	0.7391	0.8889	0.4549	0.2533	<u>0.9505</u>	0.9227	0.8968
	Recall	<u>0.9919</u>	0.7619	0.1491	0.9697	1	0.7619	0.9231	0.5878	0.9912
	F1	0.9801	0.861	0.2482	0.9275	0.6253	0.3802	0.9366	0.7384	<u>0.9417</u>
	Test time	1.7s	96.6s	Ot	7.2s	460.6s	599.0s	7.2s	474.2s	7.3s

The bold text denotes the highest performance, while underlined text indicates the second best.

anomaly dataset, PPLAD demonstrates superior anomaly detection performance compared to baseline models. For instance, PPLAD achieves an ACC score of 0.9961 and an $F1$ of 1. In contrast, COUTA records an ACC score of 0.9581 and an $F1$ of 0.8925, while DCdetector achieves 0.9872 and 0.9648, respectively. In addition, PPLAD exhibits the shortest detection time of 2.1 s, significantly outperforming TFMAE (8.1 s) and DTAAD (384.5 s), highlighting its lightweight nature, and high ACC. *Focusing on the mixed anomaly dataset*, similar trends are observed. PPLAD achieves an ACC score of 0.9929 and an $F1$ of 0.9801, outperforming competing models. Moreover, it maintains a fast detection time of 1.7 s, considerably faster than other approaches. *In summary*, PPLAD presents the efficiency and effectiveness for anomaly detection in real-world settings.

V. CONCLUSION

This article proposes PPLAD, a privacy-preserving lightweight anomaly detection model for resource-limited industrial IoT edge devices. Unlike traditional deep neural network-based approaches, PPLAD employs tiny learnable Gaussian distributions to achieve high speed, low-resource consumption, high ACC, and data privacy-preserving.

- 1) *To preserve data privacy*, the data similarity from third party secure interfaces instead of raw privacy data is used as the initial input of PPLAD.
- 2) *To achieve high speed and low-resource consumption*, a trainable Gaussian distribution with only one parameter is set for each timestamp to learn local and global similarities based on the initial similarity.
- 3) *To ensure high ACC*, an adversarial learning mechanism is designed to maximize the discrepancy between local similarities and global similarities.

To the best of authors' knowledge, this is the first work to build an industrial anomaly detection model using only trainable Gaussian distributions. Extensive experiments conducted on eight public industrial multisensor datasets and three edge devices (Thick PC, Thin Jetson Xavier NX, Mirco Raspberry Pi 4b) demonstrate that PPLAD outperforms several SOTA deep models.

Despite its advantages, the performance of PPLAD is sensitive to the ACC of the initialization similarity. Inaccurate initial similarity can significantly degrade performance. Future research will focus on developing a mechanism to address this limitation. In addition, another goal is to design a lightweight optimization

mechanism for the proposed model to automatically select appropriate parameters for different datasets.

REFERENCES

- [1] A. Villalonga, G. Beruvides, F. Castaño, and R. E. Haber, "Cloud-based industrial cyber-physical system for data-driven reasoning: A review and use case on an Industry 4.0 pilot line," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 5975–5984, Sep. 2020.
- [2] A. Sgueglia, A. Di Sorbo, C. A. Visaggio, and G. Canfora, "A systematic literature review of IoT time series anomaly detection solutions," *Future Gener. Comput. Syst.*, vol. 134, pp. 170–186, 2022.
- [3] S. Lu, J. Lu, K. An, X. Wang, and Q. He, "Edge computing on IoT for machine signal processing and fault diagnosis: A review," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11093–11116, Jul. 2023.
- [4] K. Zhang et al., "Self-supervised learning for time series analysis: Taxonomy, progress, and prospects," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 10, pp. 6775–6794, Oct. 2024.
- [5] L. Zhang, W. Bai, X. Xie, L. Chen, and P. Dong, "TMAnomaly: Time-series mutual adversarial networks for industrial anomaly detection," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 2263–2271, Feb. 2024.
- [6] L.-r. Yu, Q.-h. Lu, and Y. Xue, "DTAAD: Dual TCN-attention networks for anomaly detection in multivariate time series data," *Knowl.-Based Syst.*, vol. 295, 2024, Art. no. 111849.
- [7] C.-Y. A. Lai, F.-K. Sun, Z. Gao, J. H. Lang, and D. Boning, "Nominality score conditioned time series anomaly detection by point/sequential reconstruction," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 76637–76655, 2024.
- [8] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "Dcdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 3033–3045.
- [9] M. Ul Hassan, M. H. Rehmani, and J. Chen, "Anomaly detection in blockchain networks: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 25, no. 1, pp. 289–318, First Quarter 2023.
- [10] I. Bala, I. Pindoo, M. M. Mijwil, M. Abotaleb, and W. Yundong, "Ensuring security and privacy in healthcare systems: A review exploring challenges, solutions, future trends, and the practical applications of artificial intelligence," *Jordan Med. J.*, vol. 58, no. 2, pp. 250–270, 2024.
- [11] Z. Zhong, Z. Yu, Y. Yang, W. Wang, and K. Yang, "PatchAD: A lightweight patch-based MLP-mixer for time series anomaly detection," 2024, *arXiv:2401.09793*.
- [12] G. Sivapalan, K. K. Nundy, S. Dev, B. Cardiff, and D. John, "ANNet: A lightweight neural network for ECG anomaly detection in iot edge sensors," *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 1, pp. 24–35, Feb. 2022.
- [13] H. Xiang et al., "OptiForest: Optimal isolation forest for anomaly detection," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, 2023, pp. 2379–2387.
- [14] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023.

- [15] H. Xu, Y. Wang, S. Jian, Q. Liao, and Y. Wang, "Calibrated one-class classification for unsupervised time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5723–5736, Nov. 2024.
- [16] S. Mehnaz and E. Bertino, "Privacy-preserving real-time anomaly detection using edge computing," in *Proc. IEEE 36th Int. Conf. Data Eng.*, 2020, pp. 469–480.
- [17] B. D. D. Chen, Y. Wu, S. Tang, and Y. Zhuang, "FADngs: Federated learning for anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 2578–2592, Feb. 2025, doi: [10.1109/TNNLS.2024.3350660](https://doi.org/10.1109/TNNLS.2024.3350660).
- [18] Q. Zhang, R. Han, G. Xin, C. H. Liu, G. Wang, and L. Y. Chen, "Lightweight and accurate DNN-based anomaly detection at edge," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2927–2942, Nov. 2022.
- [19] X. Zhou et al., "Reconstructed graph neural network with knowledge distillation for lightweight anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11817–11828, Sep. 2024.
- [20] J. Pang, X. Pu, and C. Li, "A hybrid algorithm incorporating vector quantization and one-class support vector machine for industrial anomaly detection," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8786–8796, Dec. 2022.
- [21] T.-A. Nguyen et al., "Federated PCA on Grassmann manifold for IoT anomaly detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 4456–4471, Oct. 2024.
- [22] S. Ma et al., "Privacy-preserving anomaly detection in cloud manufacturing via federated transformer," *IEEE Trans. Ind. Inform.*, vol. 18, no. 12, pp. 8977–8987, Dec. 2022.
- [23] M. Abdel-Basset, N. Moustafa, and H. Hawash, "Privacy-preserved generative network for trustworthy anomaly detection in smart grids: A federated semisupervised approach," *IEEE Trans. Ind. Inform.*, vol. 19, no. 1, pp. 995–1005, Jan. 2023.
- [24] L. Cui et al., "Security and privacy-enhanced federated learning for anomaly detection in iot infrastructures," *IEEE Trans. Ind. Inform.*, vol. 18, no. 5, pp. 3492–3500, May 2022.
- [25] C. Zhang, W. Zuo, P. Yang, Y. Li, and X. Wang, "Outsourced privacy-preserving anomaly detection in time series of multi-party," *China Commun.*, vol. 19, no. 2, pp. 201–213, 2022.
- [26] Y. Fang, J. Xie, Y. Zhao, L. Chen, Y. Gao, and K. Zheng, "Temporal-frequency masked autoencoders for time series anomaly detection," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1228–1241.



Ming Li received the Ph.D. degree in computer science and technology from the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China, in 2023.

She is currently a Lecturer with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan, China. Her research interests are in the areas of parameter/state estimation, fault diagnosis, modeling, decision-making, machine learning.



Bowen Hu received the B.Eng. degree in automation in 2024 from the Hunan University of Science and Technology, Xiangtan, China, where he is currently working toward the master's degree in control science and engineering.

His current research interests include data-driven anomaly detection, deep learning, and fault diagnosis.



Haomiao Guo received the B.Eng. degree in automation from North China Electric Power University, Beijing, China, in 2022. He is currently working toward the master's degree in control science and engineering with the Hunan University of Science and Technology, Xiangtan, China.

His current research interests include data mining, deep learning, and anomaly detection.



Lei Chen (Member, IEEE) received the M.Sc. degree in computer science and engineering and the Ph.D. degree in automatic control and electrical engineering from Hunan University, Changsha, China, in 2012 and 2017, respectively.

He is currently an Associate Professor with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan, China. His current research interests include anomaly detection, lightweight design, time-series analysis, deep learning, information security of industrial control system, and Big Data analysis.



Zhaohua Liu (Senior Member, IEEE) received the M.Sc. degree in computer science and engineering and the Ph.D. degree in automatic control and electrical engineering from Hunan University, Changsha, China, in 2010 and 2012, respectively.

He is currently a Professor of Automatic Control and Systems with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan, China. His current research interests include intelligent in-

formation processing and control of wind turbines, computational intelligence and learning algorithm design, machine-learning aided fault diagnosis and prognosis.



Yepeng Xu received the B.Eng. degree in automation from the Hunan Institute of Science and Technology, Yueyang, China, in 2023. He is currently working toward the master's degree in control science and engineering with the Hunan University of Science and Technology, Xiangtan, China.

His current research interests include data-driven anomaly detection, deep learning, and fault diagnosis.